Continuous Integration

Cohort 3
Group 11
Team Aubergine
Joshua Wainwright, Piotr Koziol, Sarvesh Sridhar, Harrison Barrans, Daniel Thwaites, Callum Newton, Arnav Jamidar, Harry Turner

Methods and approaches

All code, tests, and comments are stored together in a version control system, fulfilling the continuous integration cycle as per “the repository should reliably return product source code, tests, database schema, test data, configuration files, IDE configurations, install scripts, third-party libraries, and any tools required to build the software.” [1]. To create an organised software development environment, frequent commits were encouraged, to reduce the change of conflicts caused by developers editing the same thing at the same time. Furthermore, the entire game can be compiled and started with one command, this makes it easy for developers to get set up without having to install a lot of separate things. Also, we have automated unit tests that run on a server after every commit, this verifies that the code compiles and works correctly. The server runs all major operating systems to ensure the game works for all customers and the game compiles fast because it is small, this means developers get feedback as quickly as possible. To make the non-tech members’ experience more user-friendly, the system allows the compiled game to be downloaded in a few clicks, letting non-tech team members easily try out the game.

Actual infrastructure

Gradle is a build system for Java projects which automatically downloads and installs the correct version of the Java Development Kit, along with other code dependencies, then it compiles each dependency and our own code. Depending on the arguments passed, Gradle will then either launch the game, or compile and run our automated tests.

We have created automated unit tests for various parts of the codebase, which verify that particular methods behave as described. These tests are based on JUnit, a testing library for Java code. We also use JaCoCo to produce a test coverage report, which the technical team can use to discover parts of the code which are not yet automatically tested. (More information about our testing methodology is available in the Software Testing Report.)

The team uses the Git version control system to store and track changes to all of our code, build configuration, and automated tests. The Git repository is hosted on GitHub, which is an online system that makes it available from anywhere in the world. For access control, we have a GitHub organisation which all team members are part of, so no outside people are able to access the repository.

The rest of our continuous integration tooling is based on GitHub Actions. We use a single workflow which is triggered after every commit. The workflow runs three jobs in parallel, with each instance using a different one of the three major operating systems: Windows, MacOS and Linux. Each job runs in a clean environment; this ensures that any previous actions cannot affect the outcome for a particular commit. The workflow goes through following stages:

1. Download a copy of our Git repository.
2. Install Gradle and the Java Development Kit.

3. Configure the GitHub Actions cache to store a copy of any dependencies that Gradle downloads. This reduces internet traffic for later workflow runs, which decreases the average time it takes for a developer to receive feedback on their work.
4. Run Gradle, instructing it to run our JUnit tests and produce a test coverage report using JaCoCo.
5. Upload the game executable, JUnit test report, and JaCoCo test coverage report as artifacts. This makes them available for any team member to download through the GitHub web interface. These steps attempt to run even if a previous step failed, because in some cases, a JUnit report will be generated with a more detailed explanation of the error.

No.	Paper	Home page	Date	Source
[1]	M. Fowler - “Continuous Integration”	martinfowler.com	18/01/2024	https://martinfowler.com/articles/continuousIntegration.html